#초급반 5주차

백트래킹

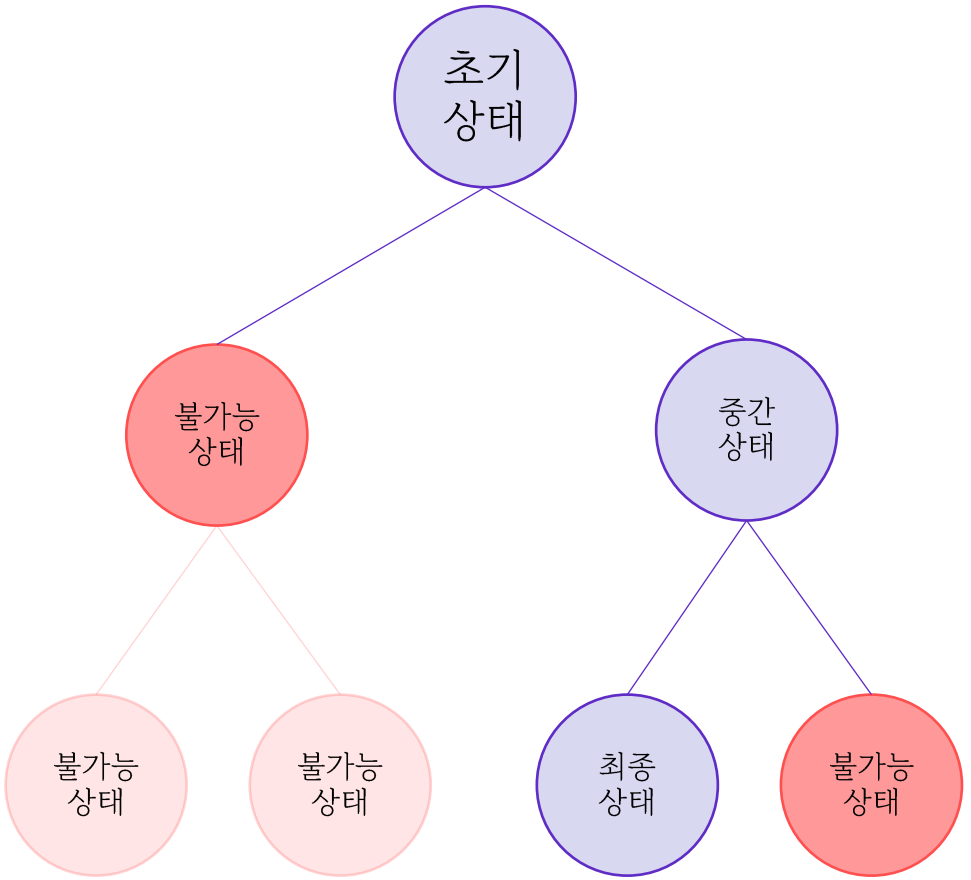
<Backtracking>

2p /<백트래킹>
4p /<예시 문제>
4p /<주사위 던지기1>
10p/<N Queen>
15p /<연습 문제>

#백트래킹
<Backtracking>

백트래킹이란, DFS를 통해 탐색을 하면서,
굳이 볼 필요 없는 경우의 수를 제거하며
문제를 푸는 방법입니다.

완전 탐색보다 탐색하는 경우의 수가 작아
시간 복잡도가 작은 장점이 있습니다.

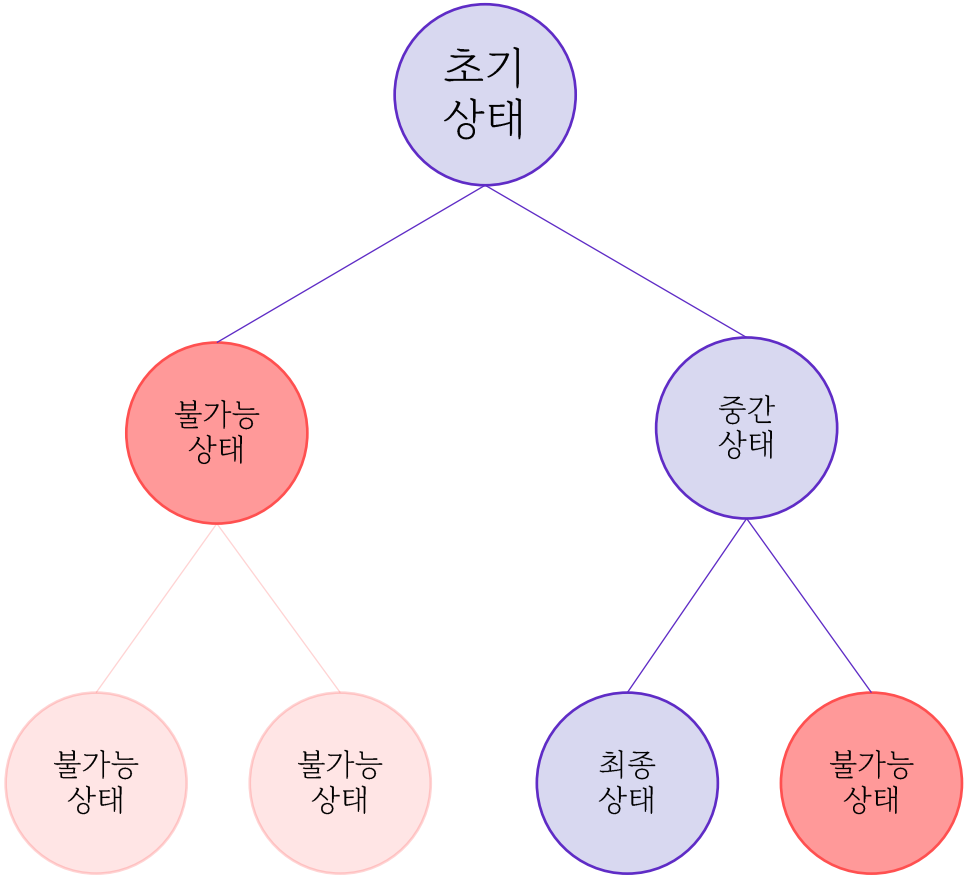


#백트래킹
<Backtracking>

예를 들어 오른쪽 경우에서는 완전 탐색을 하면 상태 7개를 모두 방문합니다.

하지만 백트래킹을 한다면 불가능 상태를 만났을 때 하위의 불가능 상태를 방문하지 않습니다. 따라서 백트래킹을 할 때에는 상태 7개 중 5개만 방문하게 됩니다.

지금은 방문하는 상태의 수 차이가 크지 않지만 불가능 상태의 수가 많다면 차이가 크게 나게 됩니다.



#백트래킹

<Backtracking> - 예시 문제

1 주사위 던지기1 (JUNGOL #1169)

<문제 설명>

- 주사위를 N번 던진다. 이때 다음을 출력한다:
  - dice1. 나올 수 있는 모든 경우의 수
  - dice2. dice1에서 중복을 제외한 모든 경우의 수
  - dice3. 모두 다른 수가 나오는 경우의 수
- 모든 경우의 수는 주사위 눈의 수의 오름차순으로 출력한다.

<제약 조건>

- $2 \leq N \leq 5$

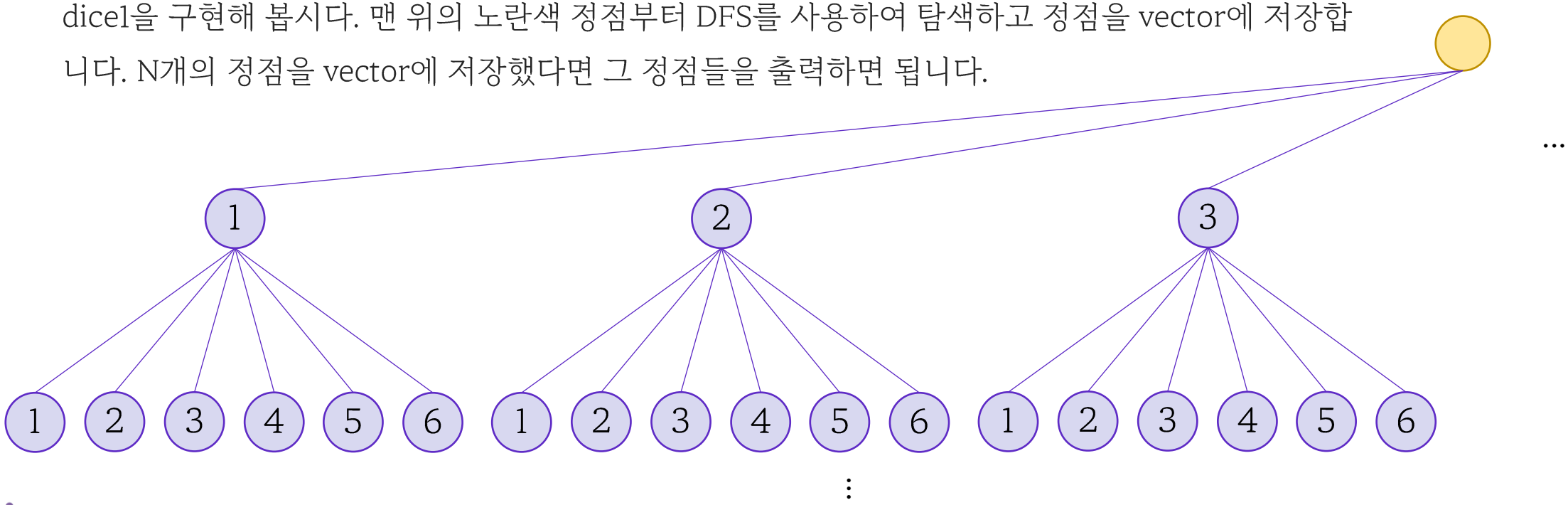
#백트래킹

<Backtracking> - 예시 문제

1 주사위 던지기1 (JUNGOL #1169)

주사위를 던졌을 때 나오는 경우의 수는 다음과 같은 트리 모양의 그래프로 생각할 수 있습니다. 정점의 숫자는 주사위를 던져 나온 숫자를 의미합니다.

dice1을 구현해 봅시다. 맨 위의 노란색 정점부터 DFS를 사용하여 탐색하고 정점을 vector에 저장합니다. N개의 정점을 vector에 저장했다면 그 정점들을 출력하면 됩니다.



#백트래킹

<Backtracking> - 예시 문제

1 주사위 던지기1 (JUNGOL #1169)

dice1 함수를 구현한 코드입니다.

i는 현재 몇 번째 주사위를 굴렸는지를 나타냅니다.

따라서 **i**가 **n+1**이라면 현재까지 굴린 주사위 수를 출력하고, 아니라면 다음 주사위를 굴립니다.

처음에는 dice1(1, n)을 호출합니다.

```
vector<int> ans;

void dice1(int i, int n) {
    if (i == n + 1) {
        for (int j = 0; j < ans.size(); j++) {
            cout << ans[j] << " ";
        }
        cout << "\n";
        return;
    }


    for (int j = 1; j <= 6; j++) {
        ans.push_back(j);
        dice1(i + 1, n);
        ans.pop_back();
    }
}
```

#백트래킹

<Backtracking> - 예시 문제

1 주사위 던지기1 (JUNGOL #1169)

dice2 함수를 구현한 코드는 다음과 같습니다.

dice1과 비슷하지만 중복된 경우를 모두 제외해야 하므로 i번째 주사위를 굴려 나온 수가 a_i라면 $a_i \leq a_{i+1}$을 만족해야 합니다.

표시한 부분이 수정된 부분입니다.

```
vector<int> ans;

void dice2(int i, int n) {
    if (i == n + 1) {
        for (int j = 0; j < ans.size(); j++) {
            cout << ans[j] << " ";
        }
        cout << "\n";
        return;
    }


    for (int j = 1; j <= 6; j++) {
        if (ans.size() > 0 && ans.back() > j) continue;
        ans.push_back(j);
        dice2(i + 1, n);
        ans.pop_back();
    }
}
```

#백트래킹

<Backtracking> - 예시 문제

주사위 던지기1 (JUNGOL #1169)

마지막으로 dice3는 다음과 같이 구현하면 됩니다. 주사위를 굴렸을 때 중복되는 숫자가 나오면 안 되므로 used 배열을 추가했습니다.

주사위를 굴려서 나오는 다음 숫자를 고를 때 이 숫자가 중복이라면 건너뛰고, 중복이 아니라면 사용 표시를 한 후 다음 주사위를 굴립니다.

```
vector<int> ans;
int used[7];

void dice3(int i, int n) {
    if (i == n + 1) {
        for (int j = 0; j < ans.size(); j++) {
            cout << ans[j] << " ";
        }
        cout << "\n";
        return;
    }


    for (int j = 1; j <= 6; j++) {
        if (used[j]) continue;
        ans.push_back(j);
        used[j] = 1;
        dice3(i + 1, n);
        ans.pop_back();
        used[j] = 0;
    }
}
```

#백트래킹

<Backtracking> - 예시 문제

주사위 던지기1 (JUNGOL #1169)

이 문제는 함수 구현 문제임에 유의합시다.

제출하는 코드에 main 함수가 포함되면 안 됩니다.

#백트래킹

<Backtracking> - 예시 문제

N Queen (JUNGOL #1889)

<문제 설명>

- 자연수 N이 주어진다.
- N×N 크기 체스판 위에, 퀸을 서로 공격하지 않는 위치에 둘 수 있는 경우의 수를 출력한다.

<제약 조건>

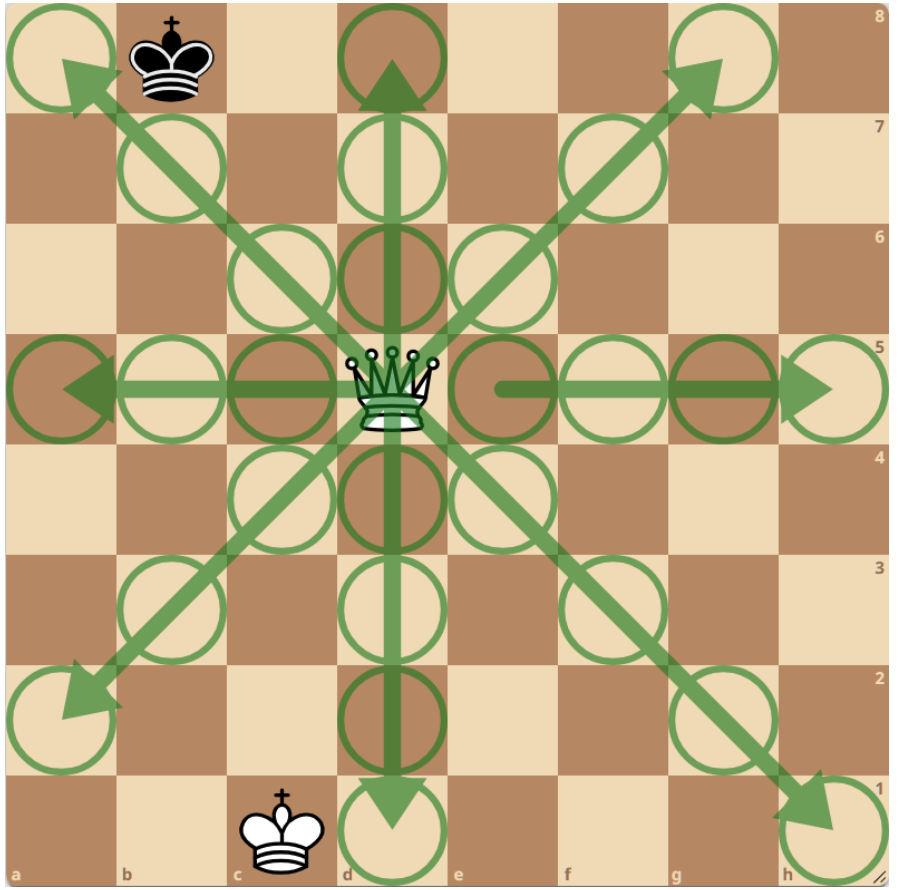
- $1 \leq N \leq 13$

#백트래킹

<Backtracking> - 예시 문제

퀸은 다음 그림과 같이 자신이 있는 위치에서 상하좌우, 대각선으로 위치한 화살표의 위의 있는 칸을 전부 공격할 수 있습니다.

초록색 동그라미로 표시된 칸이 퀸이 공격할 수 있는 칸입니다.



#백트래킹

<Backtracking> - 예시 문제

N Queen (JUNGOL #1889)

경우의 수를 줄이기 위해 몇 가지 생각을 해보겠습니다. 먼저, 한 가로줄 당 퀸을 하나씩 배치해야 함은 자명합니다.
그렇다면 현재 배치하려는 퀸이 다른 퀸과 같은 세로줄, 혹은 대각선에 위치하는지는 어떻게 확인하면 좋을까요?
세로줄의 경우는 배열에서 해당하는 칸을 `true`로 설정하는 방식으로 표시할 수 있습니다.
대각선의 경우도 비슷한 방식을 사용해볼 수 있습니다.

#백트래킹

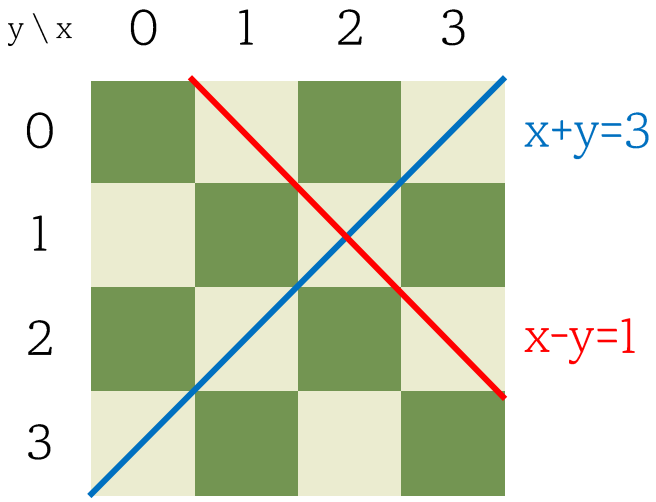
<Backtracking> - 예시 문제

N Queen (JUNGOL #1889)

같은 대각선에 속한 칸의 경우, x+y 또는 x-y의 값이 일정하게 됩니다.

따라서 퀸을 배치할 때, 각 배열에서 x+y, x-y에 해당하는 인덱스에 방문 표시를 해주면,

이후 퀸을 배치할 때 해당 칸의 대각선 방향에 퀸이 이미 배치되었는지를 확인할 수 있습니다.



#백트래킹

<Backtracking> - 예시 문제

y는 현재 배치할 퀸의 y좌표를 가리킵니다.

3개의 `bool` 배열은 각각 x, x+y, x-y 값이 겹치는지 검사하는 용도입니다.
이때, x-y는 음수가 될 수 있으므로 n을 더해 양수로 맞춰줍니다.

```cpp
#include <iostream>

using namespace std;

bool vertical[15], diagonal1[31], diagonal2[31];
int n;

int dfs(int y) {
    if (y == n) return 1;

    int result = 0;
    for (int x = 0; x < n; x++) {
        if (vertical[x] or diagonal1[x + y] or diagonal2[x - y + n]) continue;
        vertical[x] = diagonal1[x + y] = diagonal2[x - y + n] = true;
        result += dfs(y + 1);
        vertical[x] = diagonal1[x + y] = diagonal2[x - y + n] = false;
    }
    return result;
}

int main() {
    cin >> n;
    cout << dfs(0);
}
```

#연습 문제

1 주사위 던지기1 (JUNGOL #1169)

앞에서 설명한 문제입니다.

2 주사위 던지기2 (JUNGOL #1175)

주사위를 N개 던져서 합이 M이 되는 모든 경우를 찾는 문제

2 N Rook 2 (JUNGOL #6058)

룩을 배치할 수 없는 칸이 생긴다는 점만 제외하면 N Queen과 비슷한 문제

4 N Queen (JUNGOL #1889)

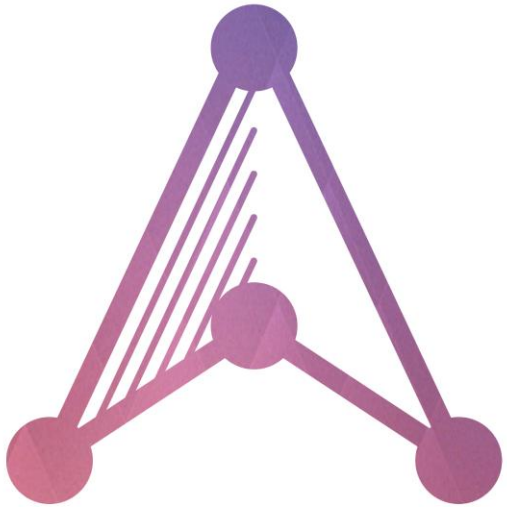
앞에서 설명한 문제입니다.

2 로또 (Lotto) (JUNGOL #2817)

K개의 수 중 6개를 고르는 순열을 출력하자

3 스도쿠 (JUNGOL #1824)

N Queen과 비슷하게 상태를 관리하는 문제



ALOHA

The algorithm club.